

Bankruptcy Prediction Using Machine Learning Classification Algorithms

Ani Musoyan

Mkrtich Arakelyan

American University of Armenia

May 2026

Abstract

Our project uses the Polish Companies Bankruptcy dataset from the UC Irvine Machine Learning Repository. We apply these classification algorithms to predict whether a company went bankrupt: k-Nearest Neighbors, Naive Bayes, Logistic Regression, Linear Discriminant Analysis, Quadratic Discriminant Analysis, Decision Tree, Support Vector Machine, and Random Forest. The dataset contains financial ratios of companies and the target variable, which tells whether a company went bankrupt or not. The dataset is highly imbalanced, so we focus on all of the necessary metrics for results. The project also illustrates preprocessing steps, for example, missing value handling, feature scaling and the treatment of extreme values. Overall, this project shows a comparative analysis of the different classification algorithms used.

Contents

1	Introduction	1
2	Dataset Description	1
3	Methodology	2
3.1	Preprocessing	2
3.2	Train-Test Split	3
3.3	Cross-Validation and Hyperparameter Tuning	3
3.4	Evaluation Metrics	4
4	Experiments	4
4.1	Models Used	4
4.2	Hyperparameter Settings	5
5	Results	6
5.1	Model Performance Tables	6
5.2	ROC Curves	9
5.3	Confusion Matrices	11
6	Comparative Analysis	13
7	Threshold Analysis	17
8	Conclusion	19

1 Introduction

The topic of bankruptcy prediction is really important in financial scenarios. It helps to identify the companies which can have financial problems in the future. So, if we predict early that company can go bankrupt or the opposite, this information is crucial for people in the financial world. It helps in decision making, for example for investors, creditors and auditors. They can understand whether the company is able to meet their financial requirements and risk tolerance levels.

The goal of this project is not only the application of different models, but also understanding how the different classification algorithms behave on financial data. Financial ratios are harder to work with which makes bankruptcy prediction more challenging than simple binary classification. So, we focus also on the model behavior besides the performance.

The dataset is very imbalanced, so bankrupt companies are only a small percentage of all observations, and non-bankrupt companies are the majority class. So in this case accuracy is misleading as the models could just always predict non-bankrupt and get high accuracy, but will fail to identify the the bankrupt companies. So, we evaluate the models by several metrics, including precision, recall, F1-score, ROC-AUC, and confusion matrix results.

We compare eight classification algorithms: k-Nearest Neighbors, Naive Bayes, Logistic Regression, Linear Discriminant Analysis, Quadratic Discriminant Analysis, Decision Tree, Support Vector Machine, and Random Forest. The rest of the report is about dataset description, preporcessing, explaining the experimental design and model training process and the presentation of results and comparative analysis.

2 Dataset Description

The dataset used for this project is the Polish Companies Bankruptcy from UC Irvine Machine Learning Repository. It is designed for bankruptcy prediction and is actually data gathered from real world events. Each observation is a company's financial statement, and the target variable represents whether the company went bankrupt or continued working.

This dataset is suitable for the project as it is a binary classification dataset, has missing values, many observations, and many financial features. The original feature names were `Attr1`, `Attr2`, ..., `Attr64`, but in our implementation, we renamed those to have more meaningful names for a more understandable analysis. The features represent financial ratios, for example profitability ratios, liquidity ratios, leverage ratios, sales-related ratios, and asset-related indicators.

The target variable, as already mentioned is binary, where class 1 indicates a company that went bankrupt, and class 0 indicates a company that did not go bankrupt during a corre-

sponding forecasting season. The dataset is very imbalanced and in every forecasting period the non-bankrupt companies are the majority while the other class is really small. This affects how the model performance should be interpreted.

The dataset is divided into five forecasting periods represented by years. The first year dataset has data from the first year of the forecasting period and the target is bankruptcy status after five years. Similar to that logic the second year dataset indicates bankruptcy after four years the third year dataset after three years, the fourth year dataset after two years, and the fifth year dataset after one year. So this allows us to see the model performances at different distances from the bankruptcy event.

The number of observations differs among the five forecasting periods. The first year dataset contains 7027 observations, including 271 bankrupt companies and 6756 non bankrupt companies. The second year dataset contains 10173 observations, including 400 bankrupt companies and 9773 non bankrupt companies. The third year dataset contains 10503 observations, including 495 bankrupt companies and 10008 non bankrupt companies. The fourth year dataset contains 9792 observations, including 515 bankrupt companies and 9277 non-bankrupt companies. Finally, the fifth-year dataset contains 5910 observations, including 410 bankrupt companies and 5500 non bankrupt companies.

As the dataset has extreme and missing values preprocessing, which is discussed in the next section is required.

3 Methodology

3.1 Preprocessing

Before starting training the data needed some preprocessing. To make the data stable and the models more reliable we needed to deal with the missing values and the financial ratios with extreme values.

For missing values, we did median imputation. So for each feature, the missing values were replaced by that feature's median value. In general, for our scenario, the median worked better as the financial ratios often contain outliers and the median is less sensitive to those. But to be sure, we also checked by trying the mean imputation too, and got unsatisfactory results which is why we kept the median in the final version. This was implemented using `SimpleImputer(strategy="median")` inside the model pipelines.

Another preprocessing step was the handling of extreme values. To reduce the role of extreme values we applied a clipping function to several models. This function simply limits the

feature values to a given range, which prevents the extreme values from causing numerical instability during training. We did this with the help of a custom a function `clip_extreme_values` and `FunctionTransformer`.

Lastly, we applied feature scaling for the models which can be affected by the scale of the input variable. Those models are k-Nearest Neighbors, Logistic Regression, Linear Discriminant Analysis, Quadratic Discriminant Analysis, and Support Vector Machine. So for these algorithms the preprocessing pipeline has this order: median imputation, extreme value clipping, standard scaling, and then model training. We did Standard scaling with `StandardScaler`, which basically makes the features have zero mean and unit variance.

For Decision Tree and Random Forest we did not do standard scaling as they split the data based on feature threshold and usually are not affected by the scale of the variables. Same for Naive Bayes as another none distance based model.

The preprocessing steps were implemented in the scikit-learn pipelines. We did it like this to avoid data leakage. Which means that the preprocessing is learned only from the training data during cross validation and then applied to the test data.

3.2 Train-Test Split

Another important part was the Train-Test Split. For our case each yearly data was divided into train and test parts using an 80–20 split. 80% of the observations are for training and 20% are left for testing and doing the final evaluations.

As our dataset is very imbalanced we used stratification. This was done to insure that both train and test splits maintain an equal distribution of our 2 classes. Without this we could have for example the whole training data have only non bankrupt companies. We did the splits separately for each forecasting period, to test the models on the same splits and have fair comparisons.

3.3 Cross-Validation and Hyperparameter Tuning

For Hyperparameter Tuning we used `GridSearchCV`. Each classification algorithm has their hyperparameters so for each we defined a set of possible hyperparameters in the model configuration file. So, the important part happens during training where `GridSearchCV` tests all combinations of the hyperparameters defined and chooses the ones which give best performance according to the chosen scoring metric.

To have a reliable tuning process we used stratified cross-validation. So we used `StratifiedKFold` with two folds, shuffling enabled, and a fixed random state. Here stratification is for the same rea-

son as in train-test split just for the cross-validation folds.

For hyperparameter selection we used the F1 score, because as mentioned earlier accuracy alone is not reliable in our case with highly imbalanced data. The F1 score is good as it balances precision and recall.

After this process where the best hyperparameters are selected, the best model pipeline is saved and then used for the test dataset. We also saved the cross-validation results, best parameter values, and final test-set metrics for later analysis.

3.4 Evaluation Metrics

To evaluate the performance of the classification algorithms we used accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrix values. These were computed on the test dataset after the best hyperparameters for the models were chosen.

Accuracy measures the proportion of correct classifications among all observations. Precision measures how many of the companies that were predicted as bankrupt are actually bankrupt, recall measures how many of the real bankrupt companies our models were able to classify as bankrupt. So, in our scenario recall is especially important as it would be very bad to miss a bankrupt company since it could cause serious problems in the financial world. Lastly, the F1 score combines precision and recall, and is very useful in the cases where a dataset is imbalanced and there is a trade off between false positives and false negatives.

We also used the confusion matrix results which show the true negatives, true positives, false negatives and false positives. This is also important for identifying which kind of errors do the models make, as for example if the models have similar accuracy scores that does not mean that they make the same kinds of errors.

Finally, the ROC-AUC was used to see if the models separate the bankrupt and non bankrupt classes good or bad across different classification thresholds. For the models which use probabilities and have probability estimates we used the probability of class 1. And for the models which have a decision function, the decision scores were used.

4 Experiments

4.1 Models Used

As mentioned before we implemented all eight classification models for all five forecasting horizons. We did this in order to compare different types of classification approaches, including

distance-based methods, probabilistic methods, linear models, discriminant analysis models, margin-based models, and tree-based methods.

This allowed us to compare not only which model performed best, but also how different model types reacted to the same financial-ratio dataset. Since the data has class imbalance, missing values, extreme ratios, and features with different scales, using several model families gave a broader view of which approaches are more suitable for bankruptcy prediction.

4.2 Hyperparameter Settings

Table 1 summarizes the hyperparameter search space that we used for the models. The same set of candidate settings was applied across the five forecasting years. The F1-score was used as the main scoring metric in GridSearchCV because the dataset is highly imbalanced, thus, accuracy alone is not sufficient for comparing bankruptcy prediction models.

Table 1: Hyperparameter settings used in GridSearchCV

Model	Hyperparameters Tuned	Values Tested / Fixed Settings
k-Nearest Neighbors	n_neighbors, metric	n_neighbors: 3, 5, 7, 11, 13, 15; metric: euclidean, manhattan
Naive Bayes	var_smoothing	var_smoothing: 10^{-9} , 10^{-8} , 10^{-6}
Logistic Regression	C, penalty	C: 0.01, 0.1, 1; penalty: l1, l2; solver fixed as liblinear; class weight fixed as balanced
Linear Discriminant Analysis	No grid-search parameters	Solver fixed as lsqr; shrinkage fixed as auto
Quadratic Discriminant Analysis	No grid-search parameters	reg_param fixed as 0.1
Decision Tree	criterion, max_depth, min_samples_split, min_samples_leaf, max_features	criterion: gini, entropy; max_depth: 5, 10, 15, None; min_samples_split: 25; min_samples_leaf: 5, 10; max_features: None, sqrt
Support Vector Machine	kernel, C, gamma	Linear kernel: C: 0.1, 1; RBF kernel: C: 1, gamma: scale
Random Forest	n_estimators, criterion, max_depth, min_samples_split, min_samples_leaf, max_features	n_estimators: 100; criterion: gini, entropy; max_depth: 10, None; min_samples_split: 2, 5; min_samples_leaf: 1, 3; max_features: sqrt, log2

The size of the search grid differed by model. Tree-based models needed more tuning because their performance depends on depth, splitting criteria, and leaf-size restrictions. LDA and QDA were kept with fixed regularization settings to improve numerical stability. After tuning, the best pipeline for each model was evaluated on the held-out test set.

5 Results

5.1 Model Performance Tables

Table 2 summarizes the best model for each forecasting year based on the test-set F1-score.

Table 2: Best performing model for each forecasting year based on F1-score

Year	Best Model	Accuracy	Precision	Recall	F1	ROC-AUC	TN	FP	FN	TP
1	Decision Tree	0.940	0.347	0.630	0.447	0.801	1288	64	20	34
2	Random Forest	0.956	0.407	0.275	0.328	0.833	1923	32	58	22
3	Random Forest	0.953	0.500	0.343	0.407	0.858	1968	34	65	34
4	Random Forest	0.931	0.330	0.301	0.315	0.827	1793	63	72	31
5	Random Forest	0.936	0.548	0.415	0.472	0.917	1072	28	48	34

The results show that tree-based models have strongest overall performance. Decision Tree achieved the highest F1-score in Year 1, while Random Forest achieved the highest F1-score in Years 2 to 5. The strongest result was obtained in Year 5, where Random Forest reached an F1-score of 0.472 and ROC-AUC of 0.917.

Table 3 presents the full model comparison for Year 5. This year is especially important because it represents prediction one year before bankruptcy, when the financial ratios are more likely to contain clearer distress signals.

Table 3: Full model performance results for Year 5

Model	Accuracy	Precision	Recall	F1	ROC-AUC	TN	FP	FN	TP
Random Forest	0.936	0.548	0.415	0.472	0.917	1072	28	48	34
Decision Tree	0.887	0.327	0.598	0.422	0.773	999	101	33	49
SVM	0.831	0.239	0.659	0.351	0.820	928	172	28	54
Logistic Regression	0.816	0.229	0.695	0.344	0.812	908	192	25	57
QDA	0.876	0.261	0.427	0.324	0.745	1001	99	47	35
KNN	0.931	0.500	0.159	0.241	0.732	1087	13	69	13
Naive Bayes	0.114	0.070	0.963	0.131	0.690	56	1044	3	79
LDA	0.928	0.385	0.061	0.105	0.809	1092	8	77	5

Random Forest achieved the best F1-score in Year 5, because it had the strongest balance between precision and recall. Decision Tree, SVM, and Logistic Regression reached higher recall than Random Forest, but their lower precision reduced their F1-scores. Naive Bayes had the highest recall, but it has very low precision which shows that it classified too many non-bankrupt companies as bankrupt.

Figures 1–5 show the F1-score comparison across all models for each forecasting year.

These figures help visualize which models achieved the best balance between precision and recall in each period.

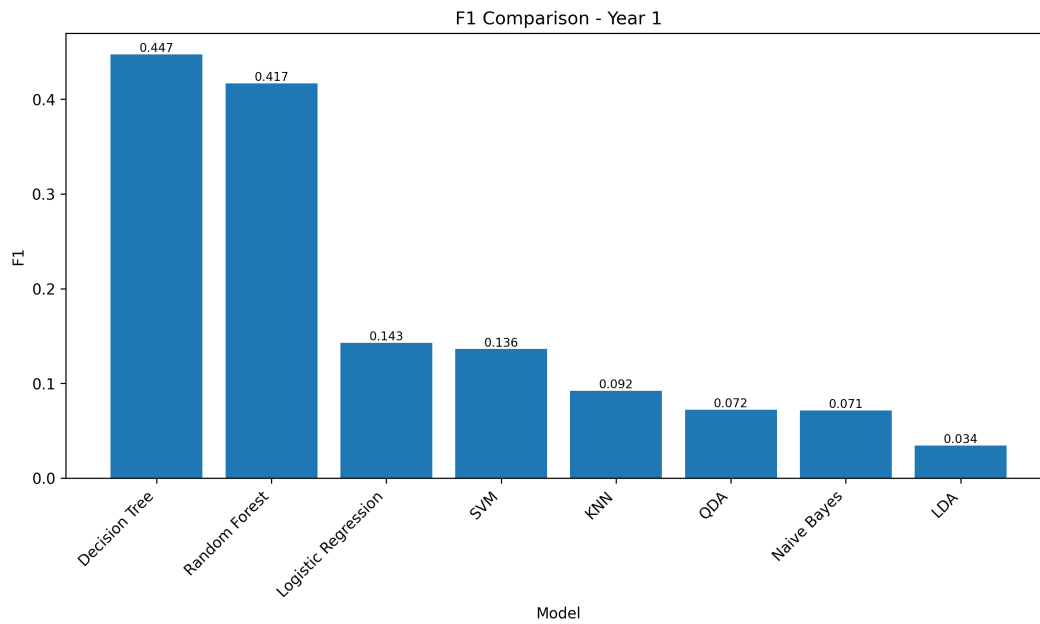


Figure 1: F1-score comparison across models for Year 1

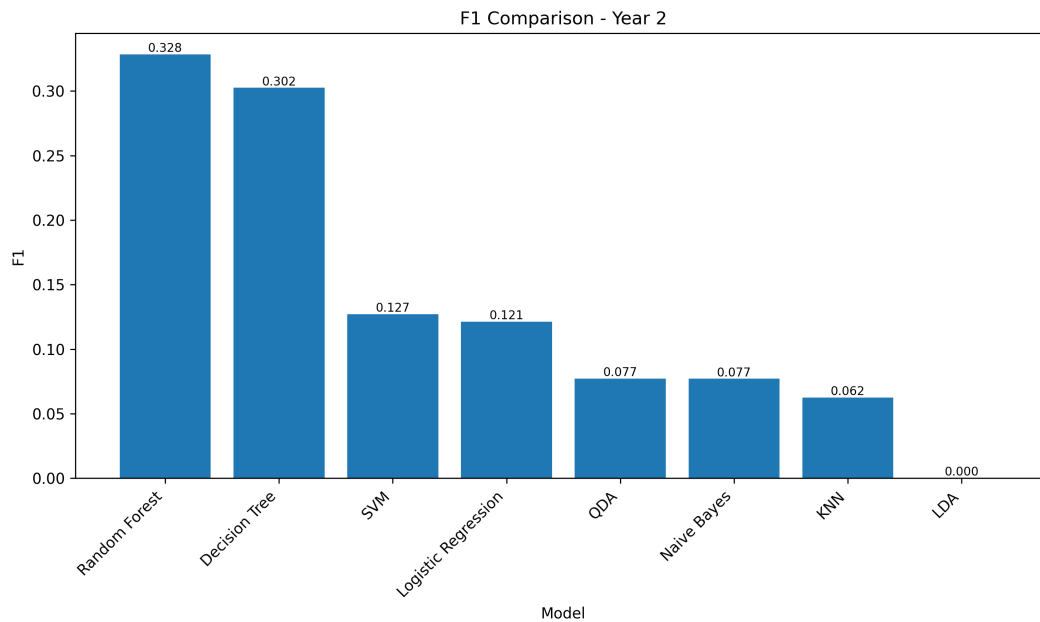


Figure 2: F1-score comparison across models for Year 2

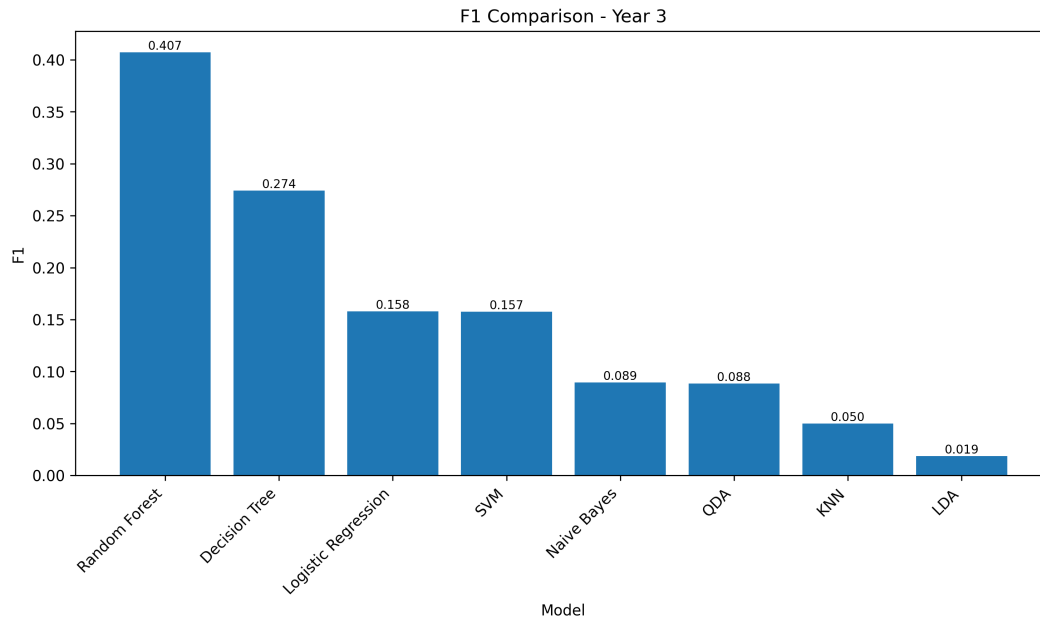


Figure 3: F1-score comparison across models for Year 3

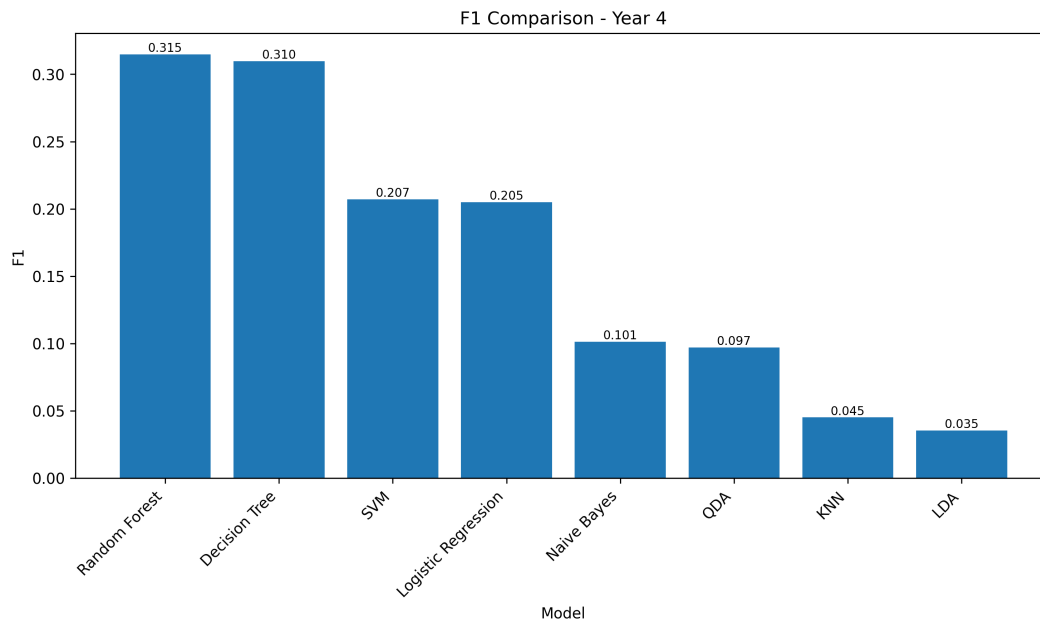


Figure 4: F1-score comparison across models for Year 4

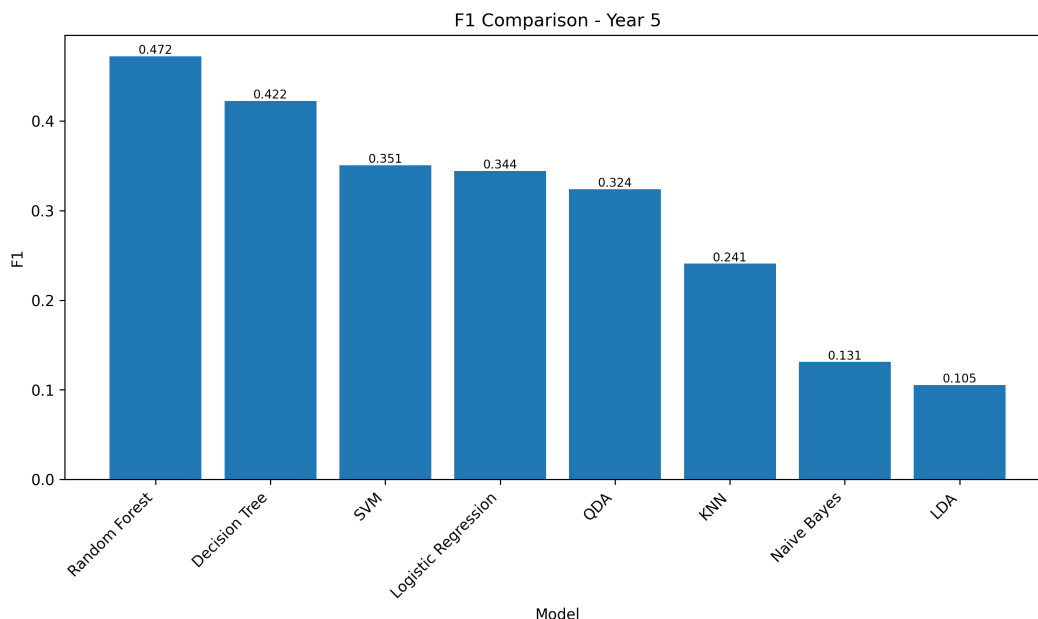


Figure 5: F1-score comparison across models for Year 5

Overall, the F1-score results show that tree-based methods were more effective than the linear, probabilistic, and distance-based models. This is likely because Decision Tree and Random Forest are better at capturing the nonlinear relationships between financial ratios and bankruptcy status. However, even the best F1-scores remain moderate, which reflects the difficulty of the task and the strong class imbalance in the dataset.

5.2 ROC Curves

We used ROC curves to evaluate how well each model separates bankrupt and non-bankrupt companies across different classification thresholds. This is especially useful for an imbalanced dataset, since it reflects ranking performance rather than relying on only one fixed classification threshold. The ROC-AUC value summarizes this behavior, where a higher value indicates better discriminatory ability.

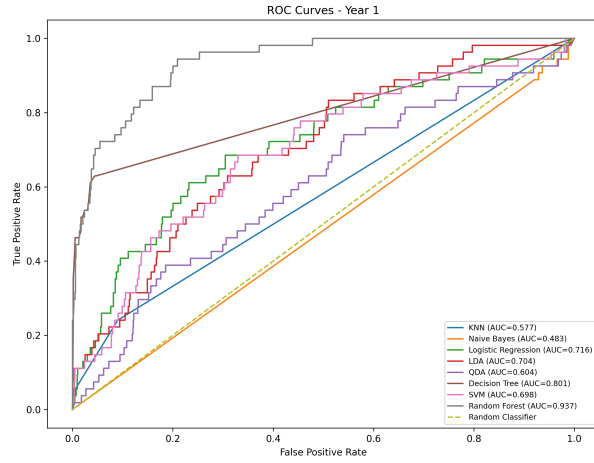


Figure 6: ROC curves for Year 1

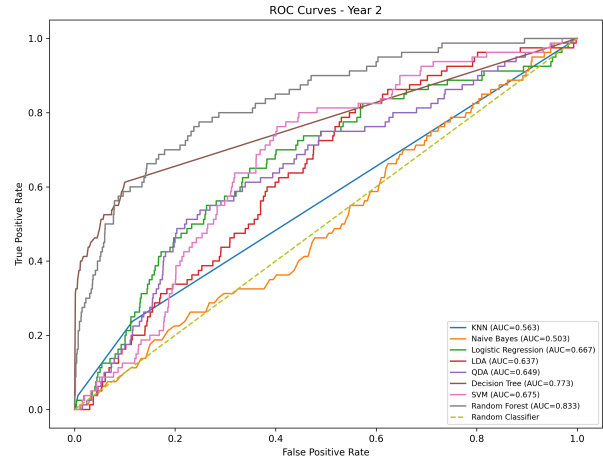


Figure 7: ROC curves for Year 2

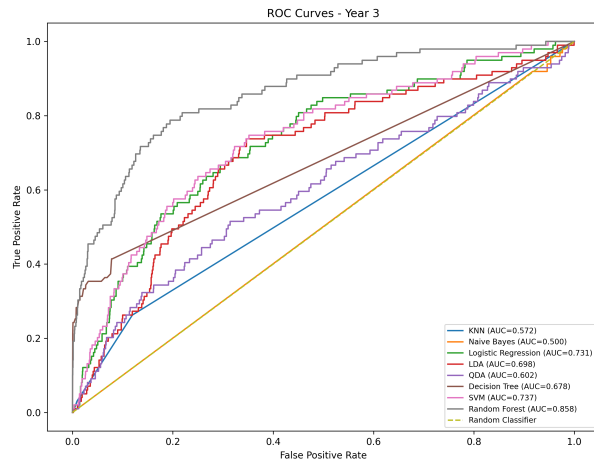


Figure 8: ROC curves for Year 3

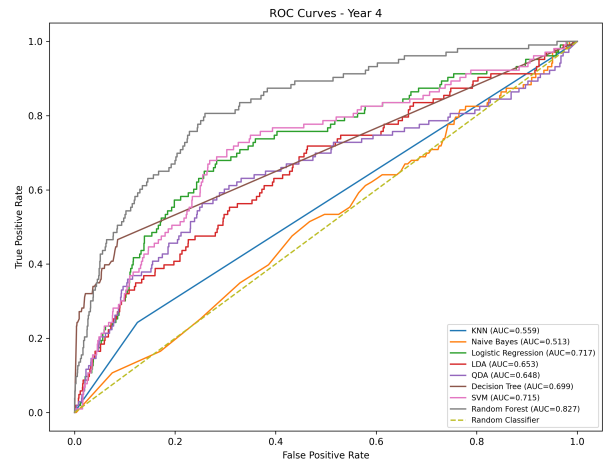


Figure 9: ROC curves for Year 4

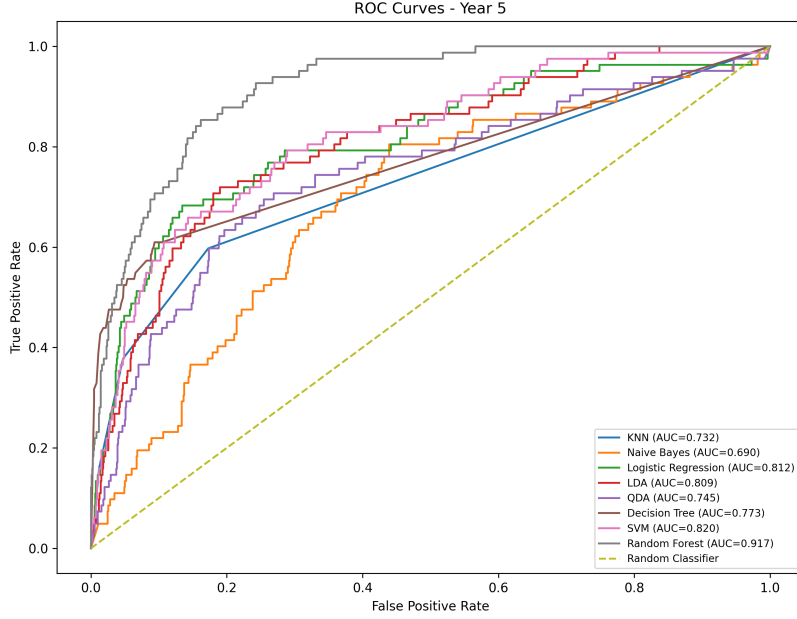


Figure 10: ROC curves for Year 5

The ROC curves show that Random Forest always had the best separation ability. In Year 1, it achieved a ROC-AUC of 0.937, which was the highest among all of our models. It also remained the strongest model in the later years, with ROC-AUC values of 0.833 in Year 2, 0.858 in Year 3, 0.827 in Year 4, and 0.917 in Year 5.

The Year 5 ROC curves are especially interesting because it represents prediction one year before bankruptcy. In this year, several models performed reasonably well, including SVM, Logistic Regression, LDA, Decision Tree, and QDA, but Random Forest still achieved the highest ROC-AUC. This supports the idea that Random Forest was the most stable model in terms of ranking bankrupt and non-bankrupt companies.

However, ROC-AUC should not be interpreted alone. Some models rank companies reasonably well and still perform poorly at the default classification threshold. Therefore, ROC-AUC was considered together with F1-score, recall, precision, and confusion matrix results.

5.3 Confusion Matrices

We used confusion matrices to examine the detailed classification behavior of the best model in each forecasting year. Unlike summary metrics such as F1-score or ROC-AUC, the confusion matrix shows the exact number of true negatives, false positives, false negatives, and true positives. This is especially useful in our bankruptcy prediction setting, where the cost of missing a bankrupt company is significant.

Figures 11–15 present the confusion matrices of the best-performing model in each year. In Year 1, the best model was Decision Tree, while in Years 2 to 5 the best model was Random Forest.

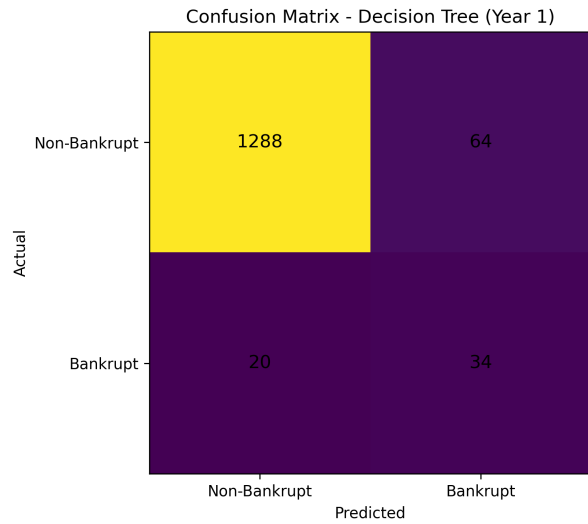


Figure 11: Confusion matrix for Decision Tree in Year 1

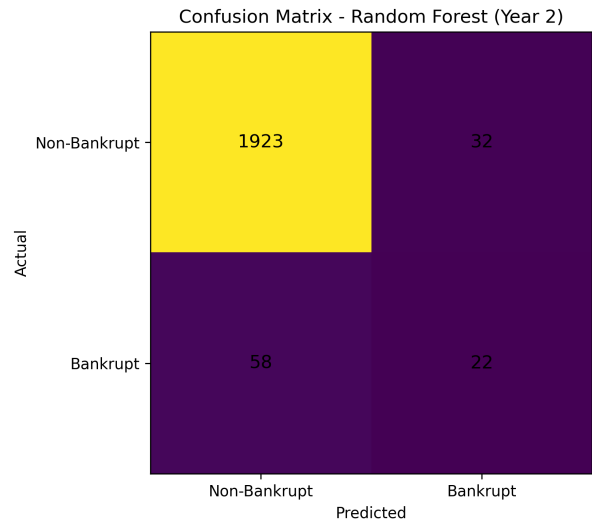


Figure 12: Confusion matrix for Random Forest in Year 2

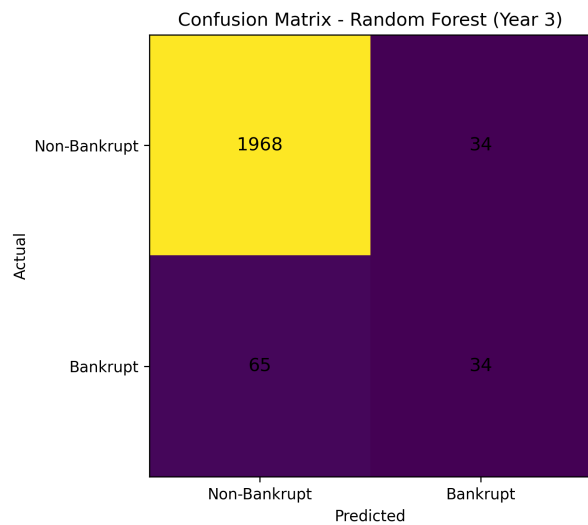


Figure 13: Confusion matrix for Random Forest in Year 3

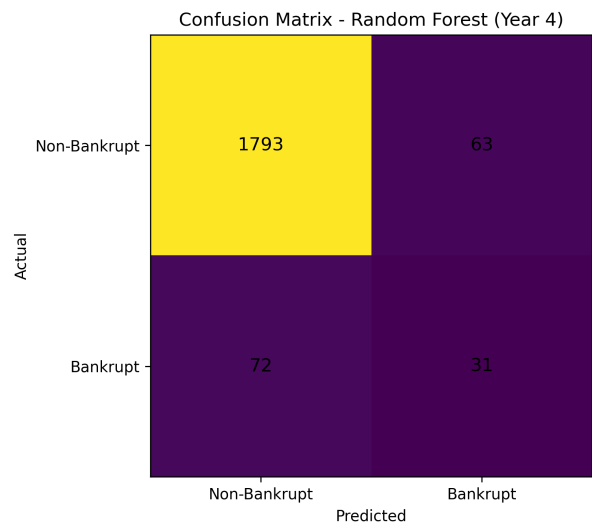


Figure 14: Confusion matrix for Random Forest in Year 4

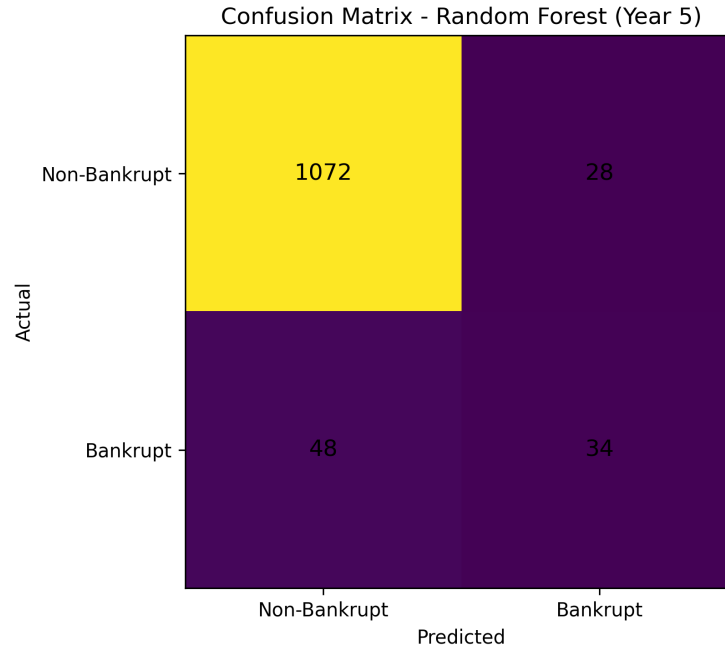


Figure 15: Confusion matrix for Random Forest in Year 5

Overall, the confusion matrices support the earlier metric-based findings and confirm that tree-based models, especially Random Forest, provided the most balanced classification performance across the five forecasting horizons.

6 Comparative Analysis

Our models comparative analysis revealed that there is a significant performance difference between model families across the five forecasting horizons. Instead of repeating the individual yearly results, in this section the main focus is on performance patterns, which models remained stable, which models favored recall over precision, and how much performance changed as the prediction horizon got closer and closer to bankruptcy.

Figure 16 shows how F1-scores changed across the five forecasting horizons. The most noticeable pattern is that tree-based models were the best performers in most years and also they were the most stable. Decision Tree was best in Year 1, and Random Forest had the highest F1-score during Years 2 through 5 and avoided the performance drops that several other models had.

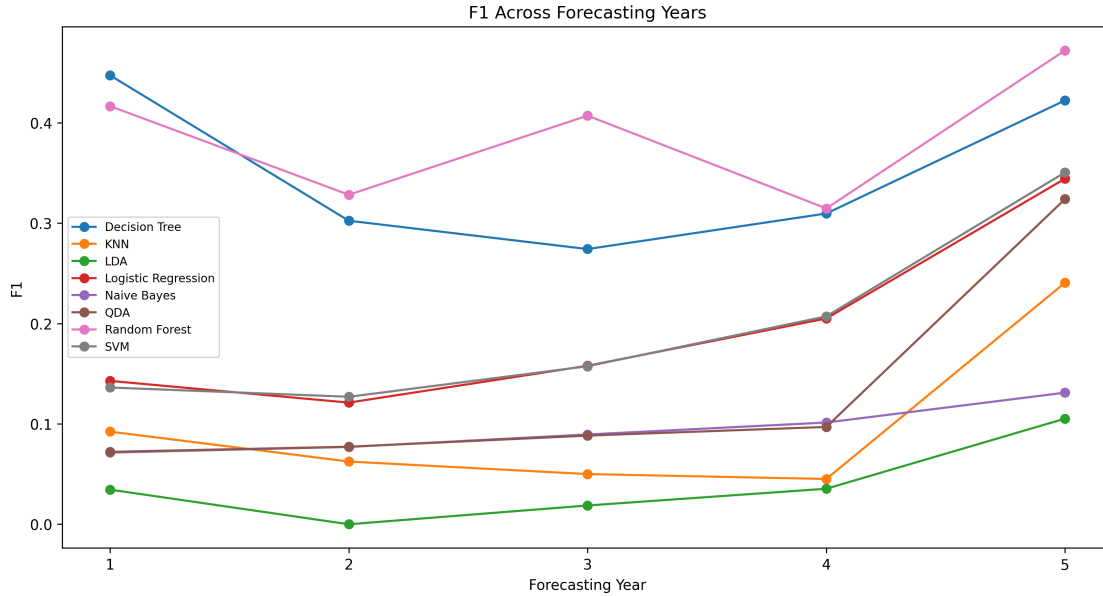


Figure 16: F1-score across forecasting years

The F1 heatmap in Figure 17 shows a short comparison of all our models for every year. It supports the notion of Random Forest and Decision Tree being the strongest models by F1-score. On the other hand, LDA, and KNN generally had weaker F1 performance. This was due to the fact that these models failed to identify enough bankrupt companies. Additionally, Naive Bayes performed poorly because it did not identify enough non-bankrupt companies.

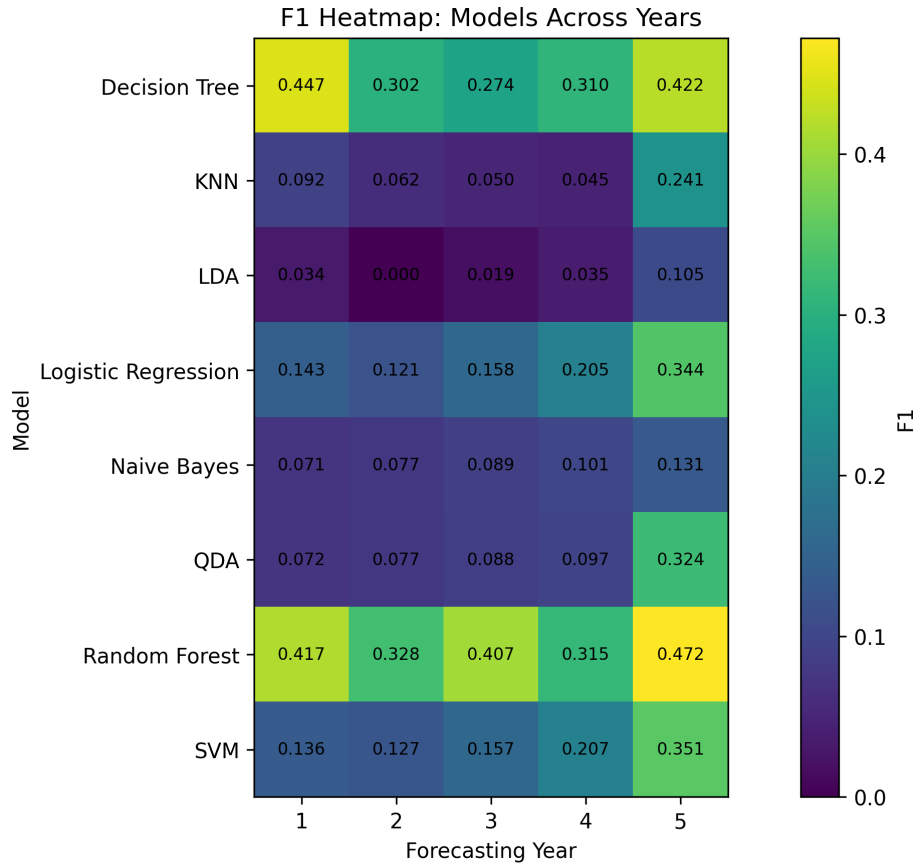


Figure 17: F1-score heatmap across models and forecasting years

The recall comparison in Figure 18 shows a different pattern. Naive Bayes and QDA achieved very high recall in several years, meaning that they identified most bankrupt companies. However, their F1-scores were much lower because they also created many false positive predictions. This shows the trade-off between catching bankrupt companies and avoiding having too many false alarms.

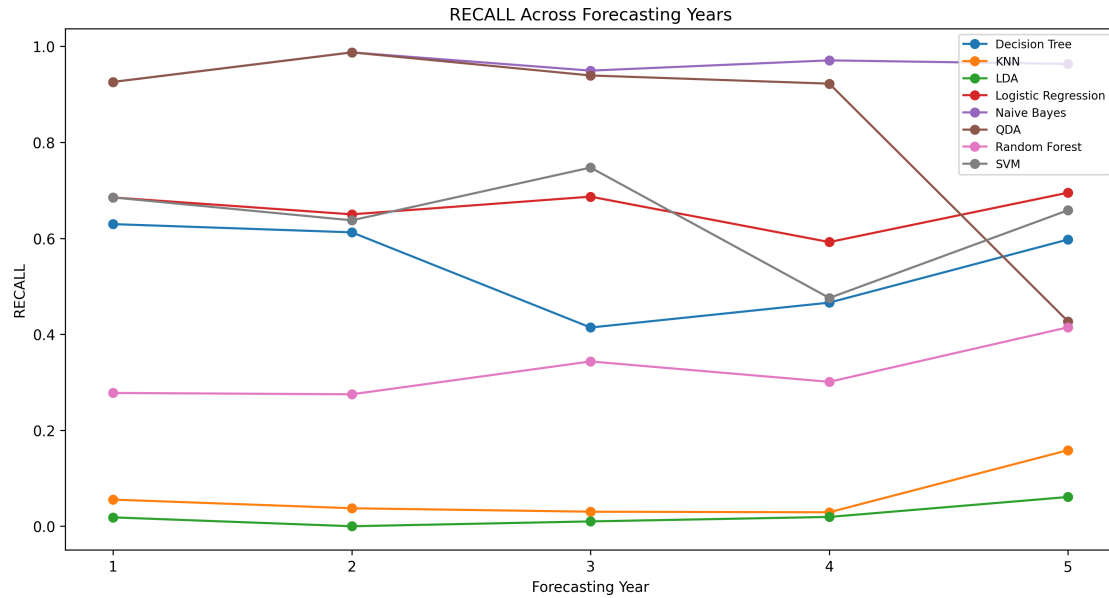


Figure 18: Recall across forecasting years

Figure 19 shows that Random Forest consistently achieved the strongest ROC-AUC across all forecasting years. This adds an important point to the F1-score results, showing that even when the final classifications were not perfect, Random Forest was still the best model at ranking companies by bankruptcy risk.

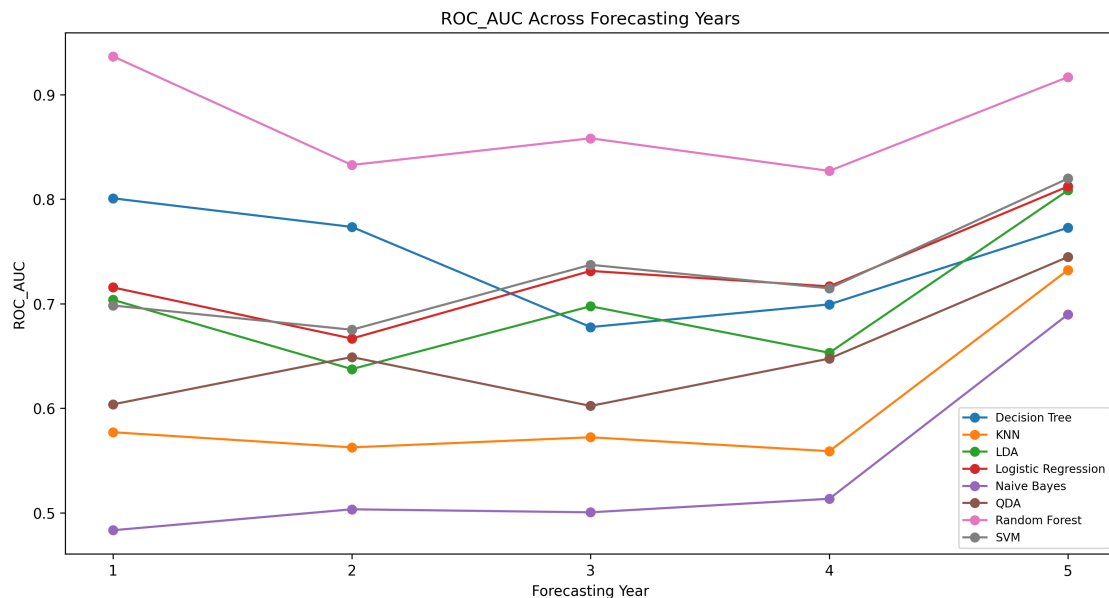


Figure 19: ROC-AUC across forecasting years

The bankrupt class percentage also changes across our 5 forecasting years, as shown in

Figure 20. The amount of bankrupt companies increases from 3.86% in Year 1 to 6.94% in Year 5. This is one of the reasons why several models performed better in Year 5, since the bankrupt class is slightly less rare, and the financial data is closer to the bankruptcy event.

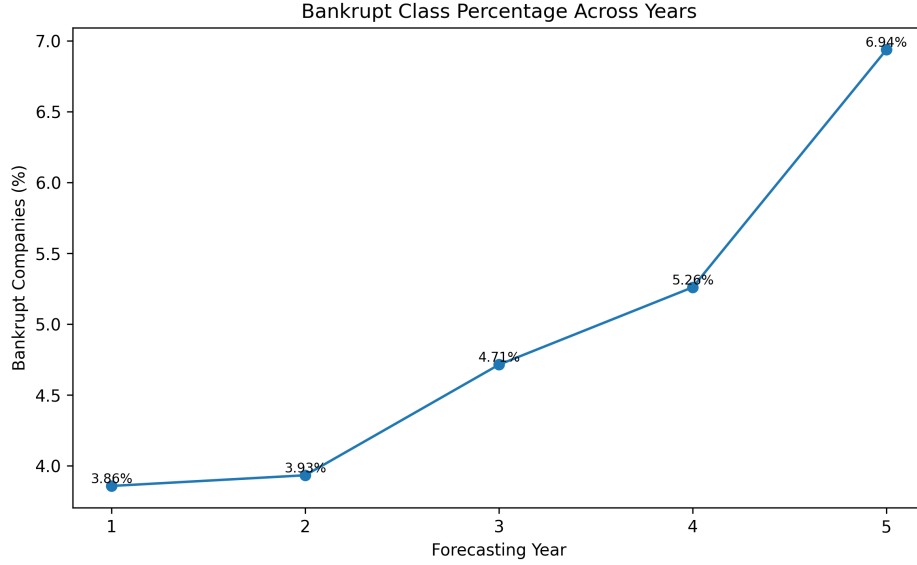


Figure 20: Bankrupt company percentage across forecasting years

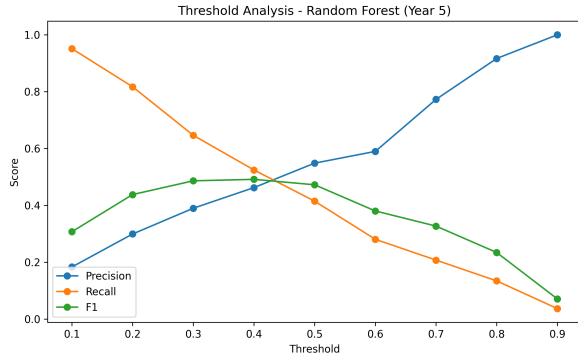
To summarize, the comparative results revealed that Random Forest was the most reliable model across the forecasting horizons. It did not consistently have the highest recall, but it provided the best balance between precision and recall, moreover, it achieved the strongest ROC-AUC values. Naive Bayes and QDA were useful for showing the importance of recall, but their low precision made them less reliable as final prediction models. Logistic Regression and SVM improved in later years, especially Year 5, but they still did not outperform Random Forest in overall balance.

7 Threshold Analysis

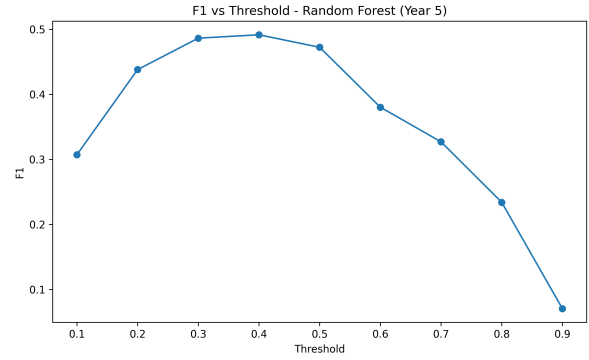
The previous sections results were mostly based on the default classification threshold of 0.5. Nonetheless, in an imbalanced bankruptcy prediction problem, this threshold may not provide the best balance between identifying bankrupt firms and avoiding false alarms. Therefore, we performed a threshold analysis on the Random Forest model, since it was the model with the strongest overall performance across the previous experiments.

We did the analysis by changing the threshold from 0.1 to 0.9 with a step size of 0.1 and calculating all evaluation metrics for each threshold. Lower thresholds classify more companies as bankrupt, which usually increases recall but reduces precision. But higher thresholds had the

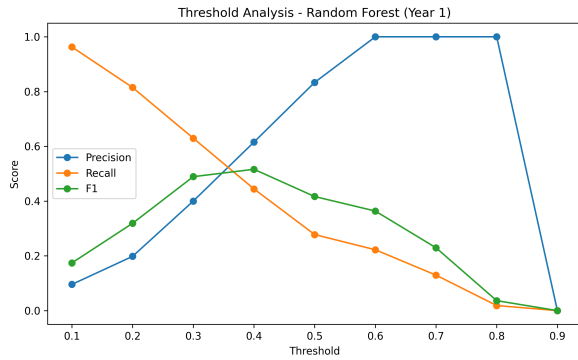
opposite effect on performance.



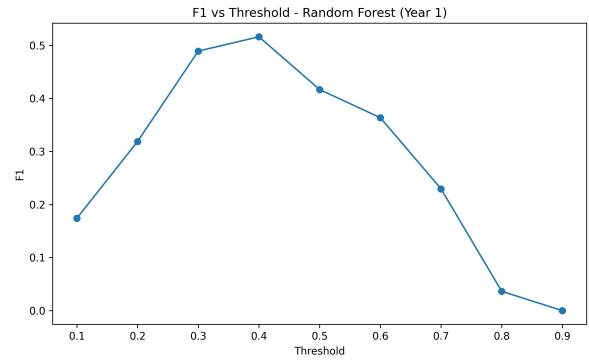
(a) Precision, Recall, and F1 across thresholds for Random Forest in Year 5.



(b) F1 score across thresholds for Random Forest in Year 5.



(c) Precision, Recall, and F1 across thresholds for Random Forest in Year 1.



(d) F1 score across thresholds for Random Forest in Year 1.

Figure 21: Threshold analysis for the Random Forest model.

Figure 21 shows that Random Forests performance changes noticeably when the classification threshold is modified. The analysis shows that a slightly lower threshold can improve F1-score in some forecasting years by capturing more bankrupt companies without increasing false positives too much.

Table 4: Best Random Forest threshold by F1-score across forecasting years

Year	Best Threshold	Precision	Recall	F1	Accuracy	Default F1	F1 Change
Year 1	0.4	0.615	0.444	0.516	0.968	0.417	+0.099
Year 2	0.5	0.407	0.275	0.328	0.956	0.328	+0.000
Year 3	0.5	0.500	0.343	0.407	0.953	0.407	+0.000
Year 4	0.4	0.310	0.427	0.359	0.920	0.315	+0.044
Year 5	0.4	0.462	0.524	0.491	0.925	0.472	+0.019

Table 4 summarizes the best Random Forest threshold for each forecasting year based on F1-score. The default threshold of 0.5 remained optimal in Years 2 and 3. However, in Years 1, 4,

and 5, the best threshold became 0.4, implying that a slightly lower threshold improved the balance between precision and recall.

The biggest improvement in performance happened in year 1 where changing the threshold from 0.5 to 0.4 increased the F1-score from 0.417 to 0.516. Minor improvements were also noticeable in years 4 and 5 when the threshold was decreased to 0.4.

The results also show that threshold selection involves a practical trade-off. A lower threshold increases the number of companies classified as bankrupt, which improves recall but can increase false positives. A higher threshold reduces false positives but may miss too many bankrupt companies. Based on F1-score, thresholds of 0.4 and 0.5 were the most suitable operating points for the Random Forest model across the five forecasting years.

8 Conclusion

In this project we evaluated different classification models for predicting bankruptcy using the Polish companies bankruptcy dataset across five forecasting years. The main challenge we faced was the task of handling the strong class imbalance. In every year, bankrupt companies represented only a small share of the observations. Because of this, accuracy alone was not a sufficient measure of model quality. The analysis therefore focused mainly on F1-score, recall, precision, ROC-AUC, confusion matrices, and threshold behavior.

The results showed that the difficulty of bankruptcy prediction depends heavily on the forecasting horizon. Predicting bankruptcy several years in advance is naturally harder because early financial ratios do not yet fully reflect the bad signals in the companies performance. This was visible in the weaker and less stable model performances we had in the earlier forecasting years. Performance generally improved in Year 5, where the data represents the period closest to bankruptcy and the financial signals are stronger. The increasing percentage of bankrupt companies across the yearly datasets also made the later years slightly less imbalanced, which helped some models improve performance.

The strongest overall pattern was that tree-based models handled the financial-ratio data better than the other model families. This probably comes from their ability to capture nonlinear relationships and interactions between financial indicators. Random Forest was the most reliable model because it did not only perform well at the selected classification threshold, but also ranked companies by bankruptcy risk more consistently across the forecasting horizons.

However, the results also show that there is no single metric that fully describes model performance. Naive Bayes and QDA often had very high recall, meaning they detected many

bankrupt companies, but their precision was very low. This means they produced too many false alarms. On the other hand, models with higher precision sometimes missed too many bankrupt companies. This trade-off is important in bankruptcy prediction because both types of errors matter, since missing a risky company is costly, but flagging too many healthy companies reduces the usefulness of the model.

The threshold analysis further confirmed this point. The default threshold of 0.5 was not always the best choice for Random Forest. In Years 1, 4, and 5, lowering the threshold to 0.4 improved the F1-score by creating a better balance between precision and recall. This shows that model performance can be improved not only by selecting a better algorithm, but also by choosing a threshold that matches the goal of the prediction task.

Random Forest was the most reliable model in this experiment because it provided the best combination of F1-score, ROC-AUC, and stability across forecasting years. More importantly, the project showed that bankruptcy prediction should be treated as a risk-ranking and decision-support problem, not just a simple classification task. A useful model should not only produce high accuracy, but should help identify financially distressed companies while keeping false alarms at a manageable level. Thus, the project shows that bankruptcy detection is not about getting high accuracy, but more about finding a model that can identify financial distress with a reasonable balance between missed bankruptcies and false alarms.

References

- [1] UCI Machine Learning Repository. *Polish Companies Bankruptcy Data Set*. Available at: <https://archive.ics.uci.edu/dataset/365/polish+companies+bankruptcy+data>
- [2] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [3] James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An Introduction to Statistical Learning: With Applications in Python*. Springer.
- [4] Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems* (3rd ed.). O'Reilly Media.